



Temperature-Humidity Monitoring System Based Automatic Fan Actuator with Thingsboard

Group 7

Names : 1. Antonio Benedictus Harianja – 2421010025
2. Kaila Whyli Titani - 2421010083

Class : 3ISA1

CEP CCIT
FAKULTAS TEKNIK UNIVERSITAS INDONESIA
TAHUN 2025

PROJECT ON

**Temperature-Humidity Monitoring System Based
Automatic Fan Actuator with Thingsboard**

Developed by

- 1. Antonio Benedictus Harianja**
- 2. Kaila Whyli Titani**

Temperature-Humidity Monitoring System Based Automatic Fan Actuator with Thingsboard

Batch Code : 3ISA1

Start Date : 3 December 2025

End Date : 24 December 2025

Name of Faculty : Listyo Edi Prabowo, ST, MT

Names of Developer:

1. Antonio Benedictus Harianja
2. Kaila Whyli Titani

Date of Submission: 24 December 2025



CERTIFICATE

This is to certify that this report titled “Temperature-Humidity Monitoring System Based Automatic Fan Actuator with Thingsboard” embodies the original work done by Antonio Benedictus Harianja and Kaila Whyli Titani. Project in partial fulfillment of their course requirement at NIIT.

Coordinator:

Listyo Edi Prabowo, ST, MT

ACKNOWLEDGEMENT

Authors would like to praise to God Almighty bless and so Authors can finish this Project 1 entitled “Temperature-Humidity Monitoring System Based Automatic Fan Actuator with Thingsboard”. Author would like to thanks to Mr. Listyo, as our faculty who has given usefull suggestion which are help Author in writing this paper.

The Author realized that this paper far from perfection, then the Author will be very unfettered the chest and was encouraged, if colleagues and the guide's agreeing lecturer gave the suggestion & criticism for this paper perfection.

SYSTEM ANALYSIS

System Summary:

The purpose of this project is to design an automated system built to operationally mitigate an environmental condition and monitor the ambient temperature and humidity of an environment. The system is built using various components to make it respond and monitor reliably and consistently.

The designed system is built using a temperature and humidity DHT11 sensor. The sensor is able to provide an ambient measurement of temperature and humidity in the surrounding atmosphere. The ESP8266 microcontroller receives and processes this sensor data and is programmed to use the temperature value that is set as a threshold for comparison. In the event the set value is exceeded, the system triggers a relay that activates a cooling fan which mitigates the excessive temperatures. Once an accepted temperature value is obtained, the fan is disengaged.

The system design facilitates the debugging aspect as it employs Serial communication to provide live system status updates alongside the sensor readings. The ESP8266 facilitates this action as it is able to use the internet to provide a cloud service using the ThingsBoard to broadcast the temperature and humidity values alongside the operational status of the fan.

The cloud service allows the users to monitor and observe the environmental values and system status in real time and participate in the system values and observe it without having to go to the system's physical location.

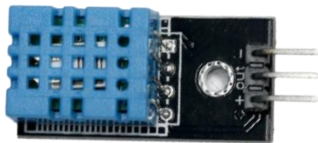
The ESP8266 microcontroller is the system's central piece of technology, and it manages the sensors and activates the relay and fan as required. With the addition of self-automation and online monitoring, the Temperature-Humidity Monitoring System streamlines the working process, minimizing hands-on control, and offering a practical answer to automation in environmental control systems.

COMPONENTS



1. NodeMCU ESP8266 (1 UNIT)

The NodeMCU is used as a microcontroller with built-in Wi-Fi, allowing it to control devices and communicate over a wireless network.



2. DHT11 (1 UNIT)

The sensor is used to monitor the temperature and humidity of the environment. The sensor sends digital data such as temperature ($^{\circ}\text{C}$) and humidity (%) to the NodeMCU for control and measurement.



3. MQ2 (1 UNIT)

The sensor is capable of detecting smoke and other flammable materials present in the environment. When the concentration of gas exceeds a certain value, the sensor creates an analog signal that NodeMCU can read for monitoring purposes or to activate a safety alert.



4. LDR Module (1 UNIT)

The LDR module detects the light intensity in the environment. The NodeMCU can determine if the area is bright or dark as its resistance changes depending on the amount of light it receives.

COMPONENTS



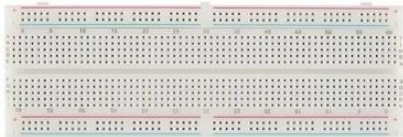
5. Single Relay Module (1 UNIT)

The single relay module acts as an electrical switch and allows NodeMCU to control high voltage appliances. In this system, the DHT11 sensor detects temperature parameters and the relay is used to turn ON or OFF the micro fan cooler.



6. Jumper Wires (Female to Male) (10 UNIT)

The jumper wires are used to provide circuits with electrical connections. The Female-to-Male jumper wires are used to connect sensors or modules directly to the pins on the NodeMCU.



7. Breadboard (1 UNIT)

The breadboard is used for prototyping and building electronic circuits without soldering, making it reusable for testing designs.



8. Jumper Wires (Female to Female) (1 UNIT)

The jumper wires serve to connect components on the breadboard or between devices to allow seamless circuit integration.

COMPONENTS



9. Micro USB (1 UNIT)

The USB data cable is used to connect the microcontroller (e.g., NodeMCU) to a computer for programming and power supply.



10. Mini Fan Cooler (1 UNIT)

When the relay triggers the mini fan cooler, it acts as the system's output actuator by turning on and enables air circulation or cooling, helping reduce the temperature.



11. Connector Adaptor (1 UNIT)

The power adapter connector is used to connect an external DC power adapter to the embedded system as the main power input. It provides a stable and reliable power supply for the ESP8266 and all connected sensors during system operation.



12. Jumper Wires (Male to Male) (2 UNIT)

The jumper wires serve to connect components on the breadboard or between devices using male pin connections, enabling reliable signal and power transmission within the circuit.

SOFTWARE

1. Arduino IDE

The Arduino IDE (Integrated Development Environment) is an open-source software designed for writing, compiling, and uploading code to Arduino-compatible boards. It provides a simple and user friendly interface for developing and programming microcontroller-based projects, supporting various libraries and functions to enhance functionality.

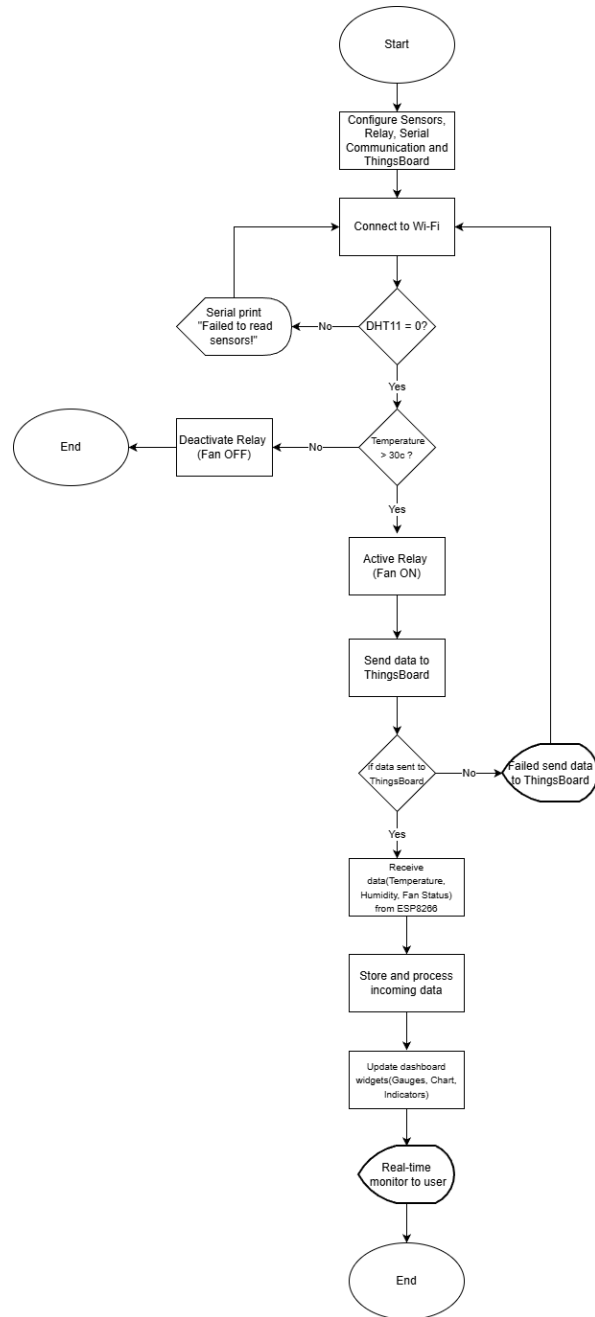


2. Thingsboard

ThingsBoard is a web-based IoT platform for collecting, processing and visualizing data from IoT devices using IoT communication protocols such as MQTT, HTTP, and CoAP, making it possible for microcontrollers such as ESP8266 to interface with it. ThingsBoard can be seen as a remote monitoring system in “Temperature-Humidity Monitoring System Based Automatic Fan Actuator with ThingsBoard” because ThingsBoard allows temperature, humidity, light intensity, and gas sensor data to be read using ESP8266 and sent wirelessly using Wi-Fi technology with either MQTT or HTTP API communications. ThingsBoard allows for a graphical interface with multiple sensor readings presented using graphs, gauges, and status indicators for real-time environmental monitoring and logging for future system analyses. ThingsBoard integration brings efficiency and usability to the system and is therefore beneficial in academic work and IoT development projects.



FLOWCHART



SOURCE CODE

```
include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#include <ArduinoJson.h>

// ===== WIFI =====
#define WIFI_SSID      "Minimal modal"
#define WIFI_PASSWORD  "12345678"

// ===== THINGSBOARD =====
#define TB_SERVER "thingsboard.cloud"
#define TB_TOKEN  "ISt4yEAvWyYHcaVoJgPs"

// ===== PIN =====
#define DHTPIN      D4
#define DHTTYPE     DHT11
#define GAS_PIN     A0
#define LDR_PIN     D3
#define RELAY_LED   D1

// ===== THRESHOLD =====
#define TEMP_LIMIT  30.0
#define GAS_LIMIT   400

WiFiClient espClient;
PubSubClient client(espClient);
DHT dht(DHTPIN, DHTTYPE);

// ===== SETUP =====
void setup() {
  Serial.begin(9600);
  delay(100);

  pinMode(LDR_PIN, INPUT);
  pinMode(RELAY_LED, OUTPUT);
  digitalWrite(RELAY_LED, HIGH);

  dht.begin();
  connectWiFi();
  client.setServer(TB_SERVER, 1883);

  Serial.println("System Ready");
}

// ===== LOOP =====
void loop() {
  if (!client.connected()) {
    reconnectTB();
  }
}
```

SOURCE CODE

```
    }
    client.loop();

    float temperature = dht.readTemperature();
    float humidity    = dht.readHumidity();

    int gasRaw = 0;
    for (int i = 0; i < 10; i++) {
        gasRaw += analogRead(GAS_PIN);
        delay(50);
    }
    gasRaw /= 10;

    int light = digitalRead(LDR_PIN);
    bool alert = false;

    if (temperature > TEMP_LIMIT || gasRaw > GAS_LIMIT) {
        digitalWrite(RELAY_LED, LOW);
        alert = true;
    } else {
        digitalWrite(RELAY_LED, HIGH);
    }

    sendTelemetry(temperature, humidity, gasRaw, light, alert);
    delay(3000);
}

// ===== WIFI =====
void connectWiFi() {
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi Connected");
}

// ===== THINGSBOARD =====
void reconnectTB() {
    while (!client.connected()) {
        Serial.print("Connecting to ThingsBoard...");
        if (client.connect("ESP8266", TB_TOKEN, NULL)) {
            Serial.println("Connected");
        } else {
```

SOURCE CODE

```
Serial.println("Retry...");

    delay(2000);

}

}

}

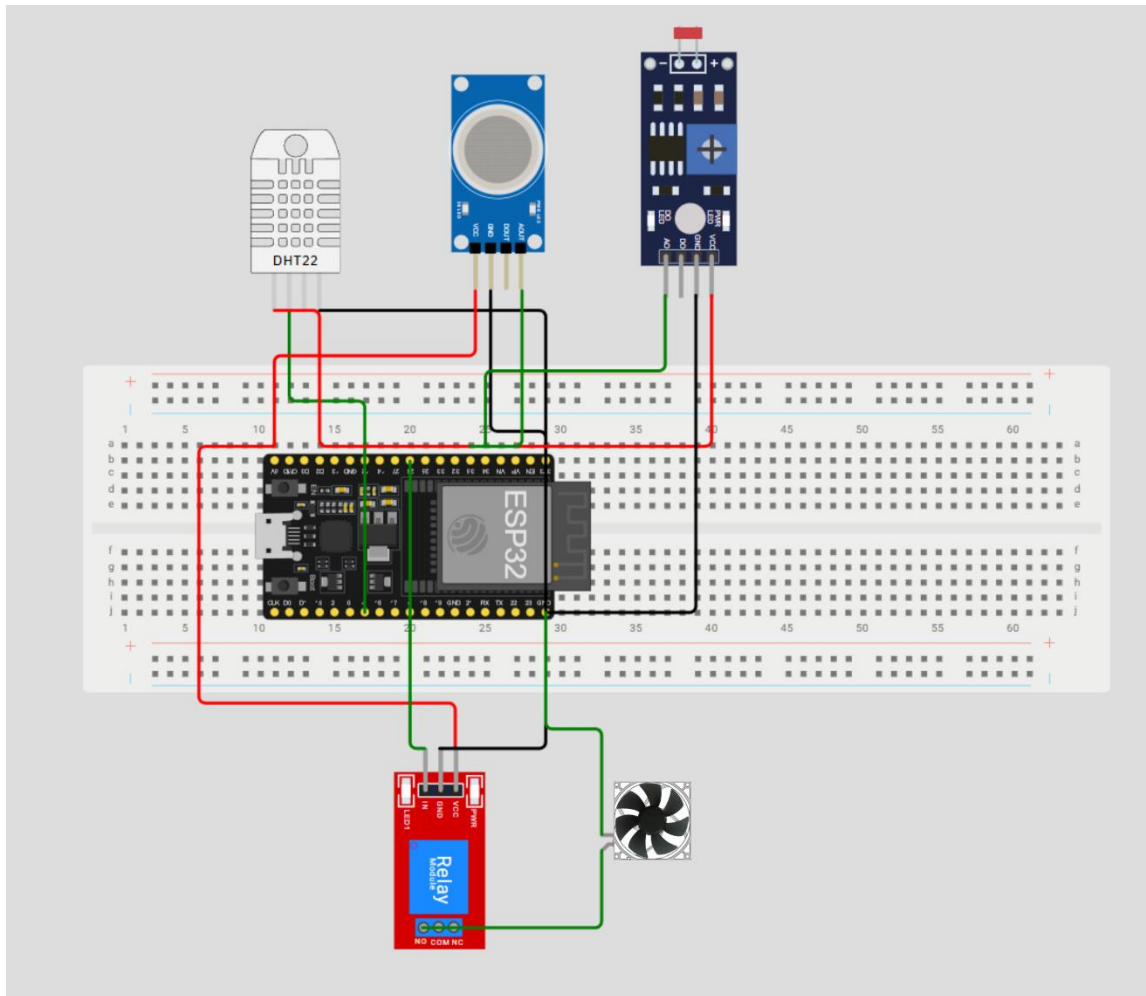
void sendTelemetry(float t, float h, int gasRaw, int light, bool alert)
{
    StaticJsonDocument<256> doc;

    doc["temperature"] = t;
    doc["humidity"]     = h;
    doc["gas_raw"]       = gasRaw;
    doc["light"]         = light == LOW ? 0 : 1;
    doc["alert_led"]     = alert ? 1 : 0;

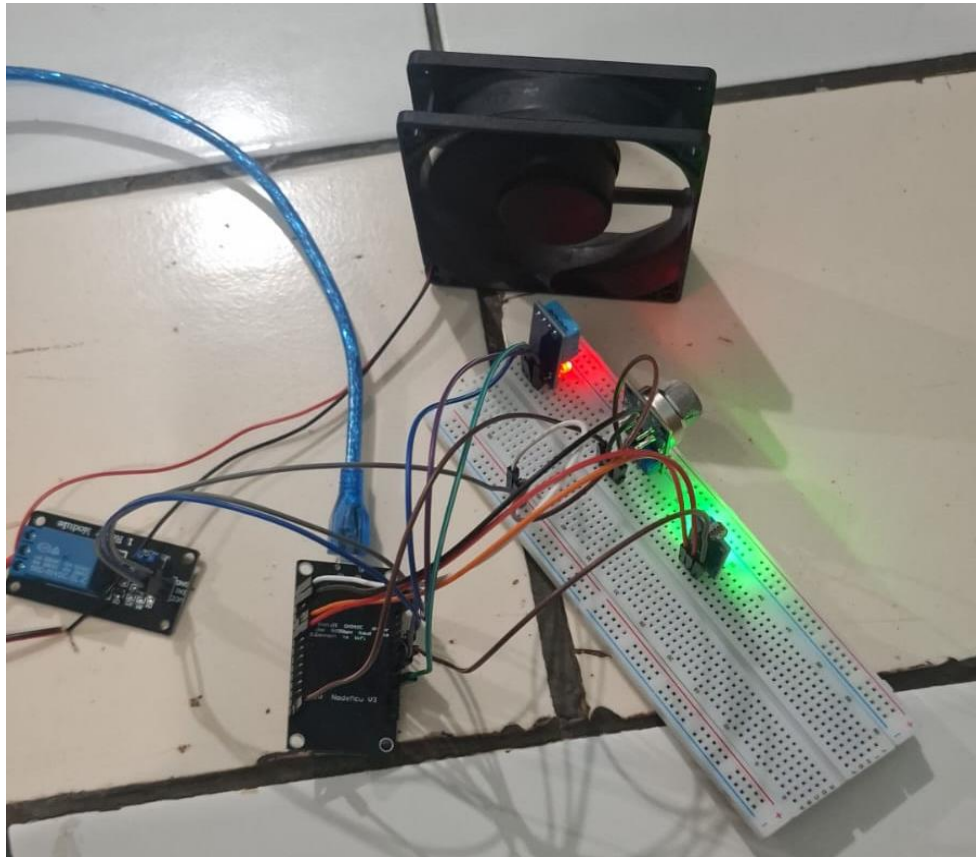
    char buffer[256];
    serializeJson(doc, buffer);
    client.publish("v1/devices/me/telemetry", buffer);

    Serial.println(buffer);
}
```

SCHEMATIC SYSTEM

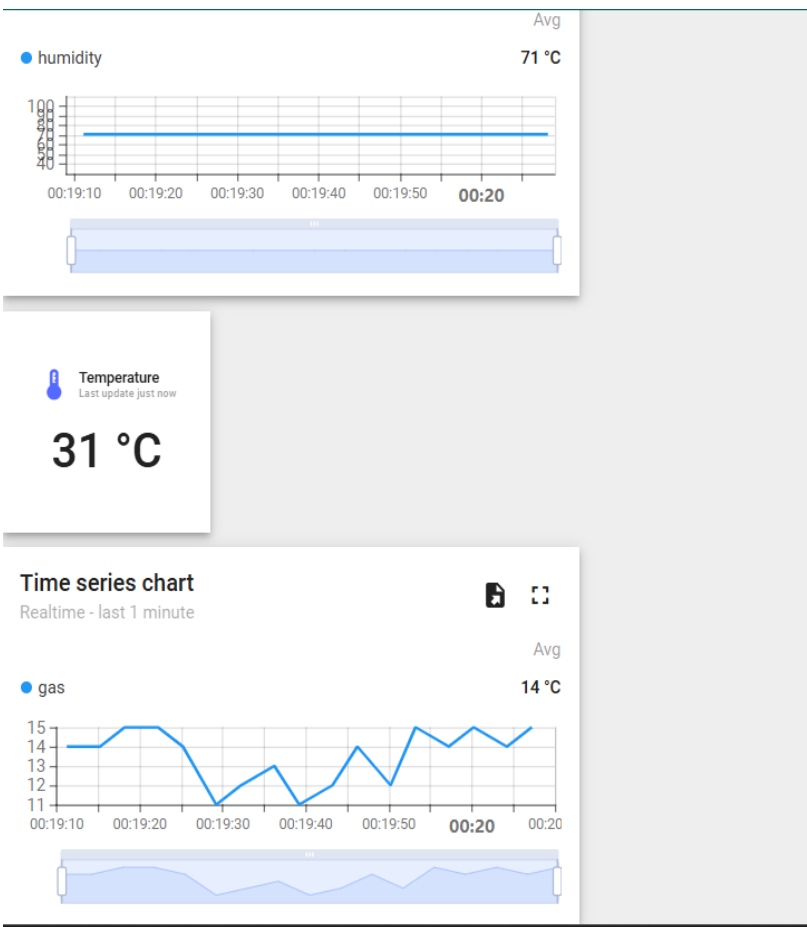


SCHEMATIC SYSTEM



OUTPUT

1. LAPTOP



OUTPUT

2. HANDPHONE

